

Main Memory

Address	Data	Comments	Address	Data	Comments
0	0000	Load 10 (1)	10000		10
1	00001	Add 13 (2)	10001		13
2	00010	Store Z (3)	10010		Z
3	00011	Load 22 (4)	10011		K
4	00100	Subt 4 (5)	10100		4
5	00101	Store in K (6)	10101		22
6	00110	(Jump to 10) (7)	10110		7
7	00111	Load 200 (8)	10111		20
8	01000	Store C (9)	11000		100
9	01001	Load K (10)	11001		200
10	01010	Subt K (11) ¹⁰¹¹⁰ K	11010		C
11	01011	Store d (12)	11011		d
12	01100	Jump to 6 (13)	11100		100
13	01101	Load 13 (14)	11101		
14	01110	Store K (15)	11110		
15	01111	Load 100 (16)	11111		
16	10000	Store C (17)			
17	10001	END (18)			
18	10000	End Program			

19	10011	0000	1101	13
20	10100	0000	1010	10
21	10101	0000	0000	Z
22	10110	0000	0000	K
23	10111	0000	0100	4
24	11000	0001	0110	22
25	11001	1100	1000	200
26	11010	0000	0000	C
27	11011	0000	0000	d
28	11100	0110	0100	100
29	11101	0000	0000	0
30	11110	0000	0000	0
31	11111	0000	0000	0

End All

Handwritten signature

Micro Memory

Address	CS code	Select	Next Addr	Comments
00000	00006	0	00001	Fetch 1 → Fetch 2
00001	00001	0	00001	Fetch 2 → Fetch 3
00010	00010	1	00000	Fetch 3 → Opcode Map
00011	00010	0	00001	Load 1 → Load 2
00100	01001	0	00000	Load 2 → Fetch 1
00101	01110	0	00111	Store 1 → Store 2
00111	01111	0	00000	Store 2 → Fetch 1
01000	10000	0	01001	Add 1 → Add 2
01001	10001	0	00000	Add 2 → Fetch 1
01010	10110	0	01011	Sub 1 → Sub 2
01011	10111	0	00000	Sub 2 → Fetch 1
01100	11100	0	00000	Jump 1 → Fetch 1
01101	00000	0	00000	Space
01110	11110	0	00000	Jump 1 → Fetch 1
01111	00000	0	00000	JEQ → Fetch 1
10000	00100	0	00000	

Planning .
 Implementing .
 Simulating .
 Presenting .

- Can we do ~~an~~ a larger instruction frame?
 - Do we need to? A lot of empty spots for Mem Dump

• Microsequencer in Control Path

• LOAD, STORE, ADD, SUBTRACT, JUMP, + SEQ (+Flag)

• Need a Memory Dump Module

Instruction Set
 + Data Path
 + State Diagram
 Stay Same

• Short Code Sequence that operates every instruction in your set at least once.

• RTL Code

Fetch Sequence
 Instruction Sequences

RTL for LOAD

Fetch 1, Fetch 2, Fetch 3

Fetch 1 → ~~PC~~

$AR \leftarrow PC$

Fetch 2

$DR \leftarrow m[AR], PC \leftarrow PC + 1$

Fetch 3

$IR \leftarrow DR[7..6], AR \leftarrow DR[6..0]$

<u>LOAD</u>	<u>STORE</u>	<u>ADD</u>	<u>SUBTRACT</u>	<u>JUMP</u>	<u>SEQ</u>
001	001	010	011	000	101

In:

AC → Accumulator
 AR → Address Register
 PC → Program Counter
 DR → Data Register
 IR → Instruction Register
 M → Memory

ALU →
 buffers →

• Fetch from main memory
 • Decode instruction
 • Execute instruction

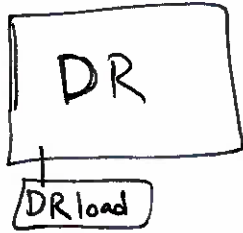
3 bits for Instruction Code
 5 bits for Address
Program

Memory
 5 bits

LOAD

Data from Memory to Data Register

Load \$DR MemAddress



- ~~Load~~ Address to AR
- ~~Read~~ ~~Data~~ in Memory
- ~~enable~~ Memory Bus
- ~~Load~~ ~~Memory~~ Bus
- ~~Disable~~ ~~Memory~~ Bus
- Load Data Register
- Disable Memory Bus

STORE

Data from Data Register to Memory address

- Disable MemBus (6), Disable PCbus (7)
- Enable DR BUS (8)
- Load membus w/ DR (9)
- ~~Disable DR BUS~~
- Set mem write (9)
- Stop mem write (10)
- Stop DR BUS (11)
- enable UF (1)
- Load AR w/ address (2)

Add

• Add two values #
• add \$dest \$val1addr \$val2addr

- Buffer 1 + Buffer 2 (1)
- enable UF (1)
- Load address w/ address 1 (Val1) (2)
- Enable membus (3)
- Set Mem read Address 1, (4)
- Enable membus (5)
- Load Buffer 1 (5)
- Disable membus (6)
- Load AR w/ address 2 (Val2addr) (2)
- Enable membus (3)
- Set Mem read (4)
- Load Buffer 2 (12)
- Disable membus (6)
- Enable ALU BUS (13)
- ~~ALU Add~~

- enable UF (1)
- Load AR w/ Destination (2)
- ALU ADD (14)
- Set Mem write (15)
- Disable ALU BUS (16)

Subtract

- Subtract two values
- Buffer 1 - Buffer 2
- Sub \$dest \$addr1 \$addr2

~~enable UF2~~ ②

- Load AR w/ address ②
- Enable membus ③
- Set mem read ④
- Load Buffer 1 ⑤
- Disable membus ⑥
- ~~enable UF3~~ Load AR w/ address ⑦

- Enable membus ③
- Set mem read ④
- Load Buffer 2 ⑫
- Disable membus ⑥
- Enable ~~ALU~~ ALU BUS ⑬

- ~~ALU subtract~~ enable UF1 ①
- Load AR w/ Destination ~~Reg~~ Addr ②
- ALU subtract ⑬
- Set mem write ⑮
- Disable ~~ALU~~ ALU BUS ⑯

Jump

- Jump to a different spot in Program Counter

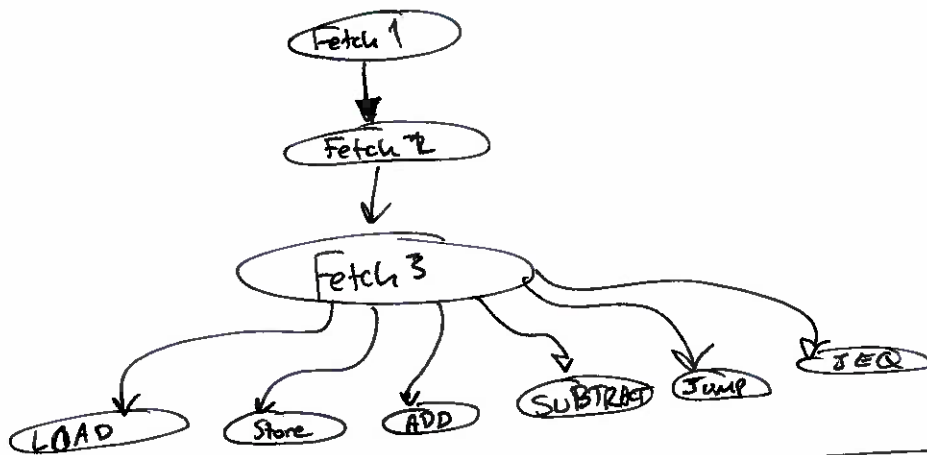
• PC ← DR [4...0]

• Jump \$PCval

jmp \$state
~~\$place~~

- ~~enable UF1~~ Load DR ~~with user field 1~~ ⑤
- Enable DRBUS ⑧
- Load PC ⑯
- Disable DRBUS ⑪

LOAD	IR ₂ IR ₀ 000	
STORE	001	
ADD	010	
SUBTRACT	011	
JUMP	100	
JEQ	101	

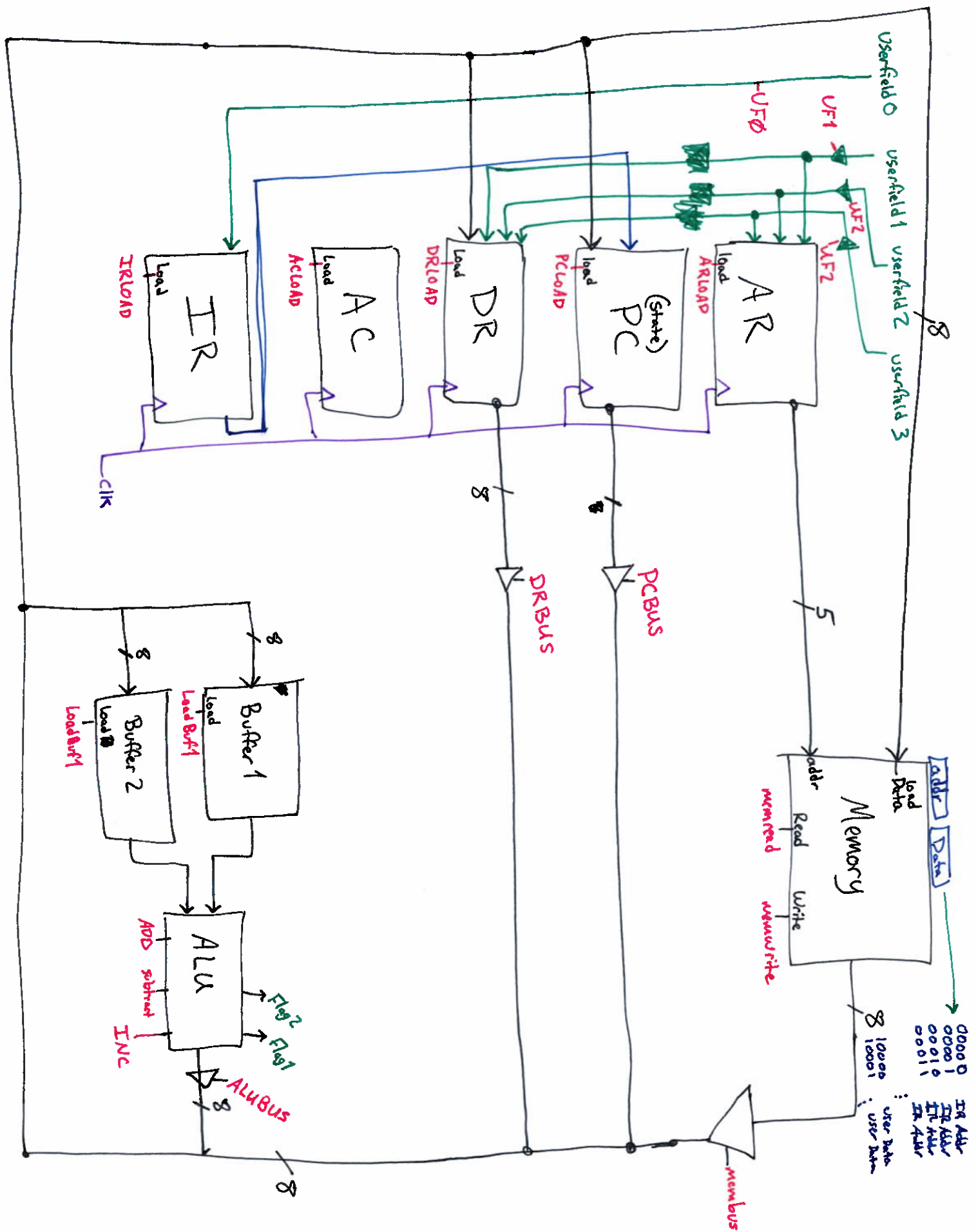


LOAD

load \$d \$m[AAAAA]

~~PC ← PC~~

PC



LOAD

• Load Address to AR from user field

• enable UFB 1 ①

• Load Address to AR ②

• ~~Set Memory Read~~

• enable membus ③

• Set memory Read ④

• Load Data Register ⑤

• Disable Mem Bus ⑥

① $Z = 10 + 13$

1 Load 10 1
2 Add 13 2
3 Store Z 3

② $K = 22 - 4$

4 Load 22 4
5 Subt. 4 5
6 Store K 6

③ $b = K + Z$

7 Load K
8 Add Z
9 Store b

④ JMP to 6

10 Jump to 6 (10)(6) 7

⑤ ~~11~~ $C = 200$

11 Load 200 8
12 Store C 9

⑥ $d = K - Z$

13 Load K 10
14 Subt Z 11
15 Store d 12

⑦ JEQ 10

16 JEQ 16 (10) 13

⑧ ~~C = 10~~

17 Load 10
18 Store C

⑨ ~~b~~ $K = 13$

19 Load 13 14
20 store ~~b~~ K 15

⑩ $C = 100$

21 Load 100 16
22 Store C 17

⑪ End

23 End 18

JEQ

- 1 $Z = 10 + 13$ (23)
 - 2 $K = 22 - 4$ (18)
 - 3 $b = K + Z$ (41)
 - 4 Jump to 6
 - 5 $C = 200$ (C=200)
 - 6 $d = K - Z$ (-5)
 - 7 ~~JEQ~~ JEQ 10
 - 8 $C = 10$
 - 9 $C = 12$
 - 10 $C = 100$ ← $C = 100$
- User function

$$Z = 10 + 13$$

$$K = Z - 4$$

$$K = K - 22$$

$$Z = 0$$

$$Z = 1$$

$$b = 7$$

$$b = 20$$

Address

00000
00001
00010
00011

Main Mem

5-bits

01000

01001

01010

01011

01100

01101

01110

01111

~~0~~10000

8-bits

100000 1010

0000 1101

13

0000 0000

Z

0000 0000

K

0000 0100

4

0001 0101

22

7

20

b

User program

000	01000	// Load 10
010	01001	// Add 13
001	01010	// Store Z
000	01010	// Load Z
011	01100	// Minus 4 (Z-4)
001	01011	// store in K
000	01011	// Load K
011	01101	// K-22

~~100~~
100

Report Flow

1. Decide on the instruction set and required extension
 2. Decide on the instruction frame
 - Op-codes bits
 - address/immediate fields
 3. Generate RTL-level behavior for the CPU fetch cycle and each instruction
 4. Define the mapping from instruction op code to Microsequencer execute
 5. branch starting address
 6. Implement and verify individual modules before full system integration
 6. Implement the control path, datapath, memory, ALU, and memory dump modules
 7. Write a short test program that uses every instruction at least once
 8. Simulate the CPU active period and memory dump period, then annotate waveforms.
-

00000
Fetch 1
 $AR \leftarrow PC$
 • PC Bus ✓
 • AR load ✓

Fetch 2
 $DR \leftarrow m[AR]$
 • PC INC $PC \leftarrow PC + 1$
 • Mem bus
 • DR load
 • Mem read

Fetch 3
 • DR BUS $IR \leftarrow DR[7:5]$
 • IR load $AR \leftarrow DR[5:0]$
 • AR load

000
Load 1
 • Mem bus
 • Mem read
 • DR load
 $DR \leftarrow m[AR]$
Load 2
 • DR bus
 • AC load
 $AC \leftarrow DR$

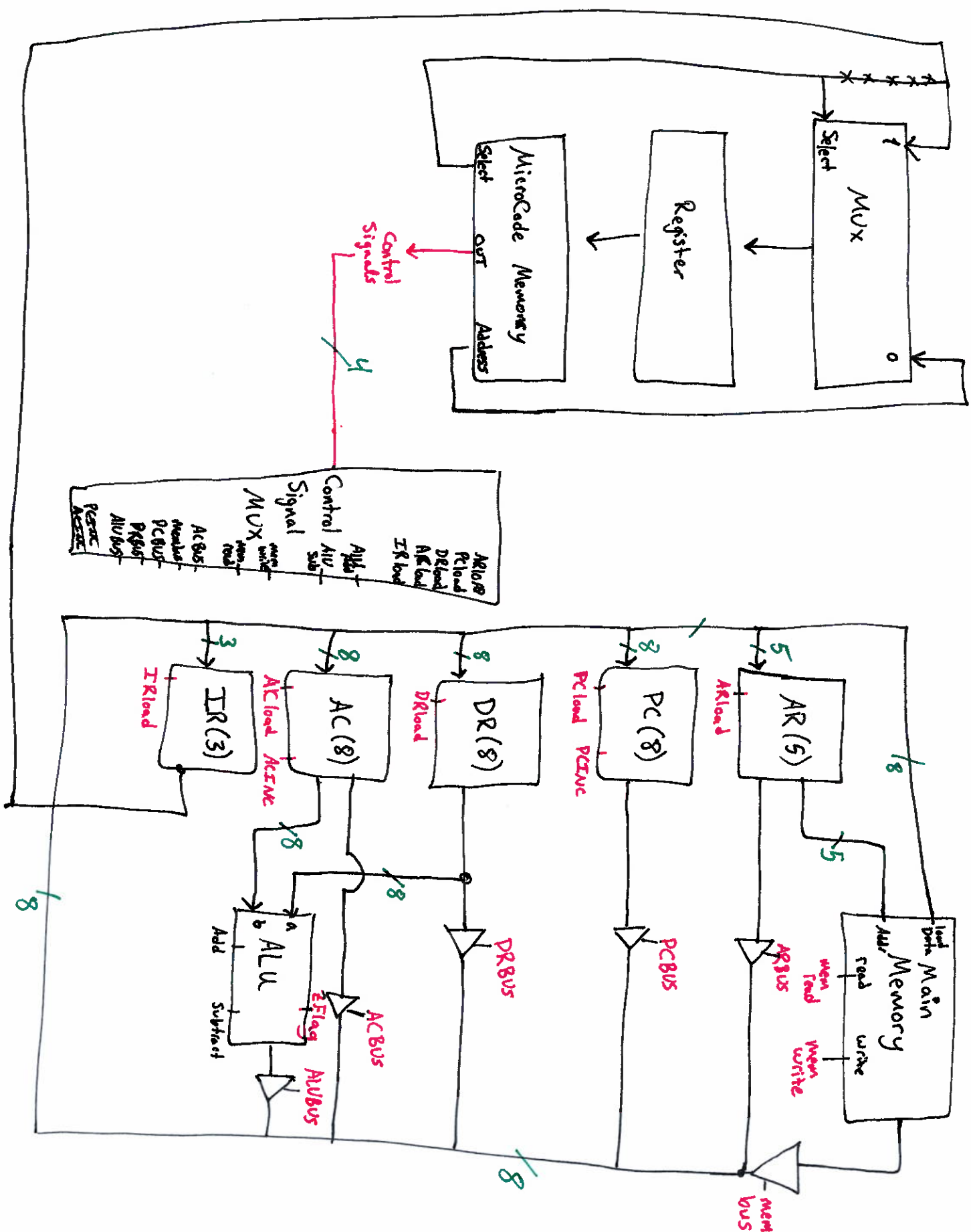
Jump 1
 • PC load AR
 • $PC \leftarrow AR$

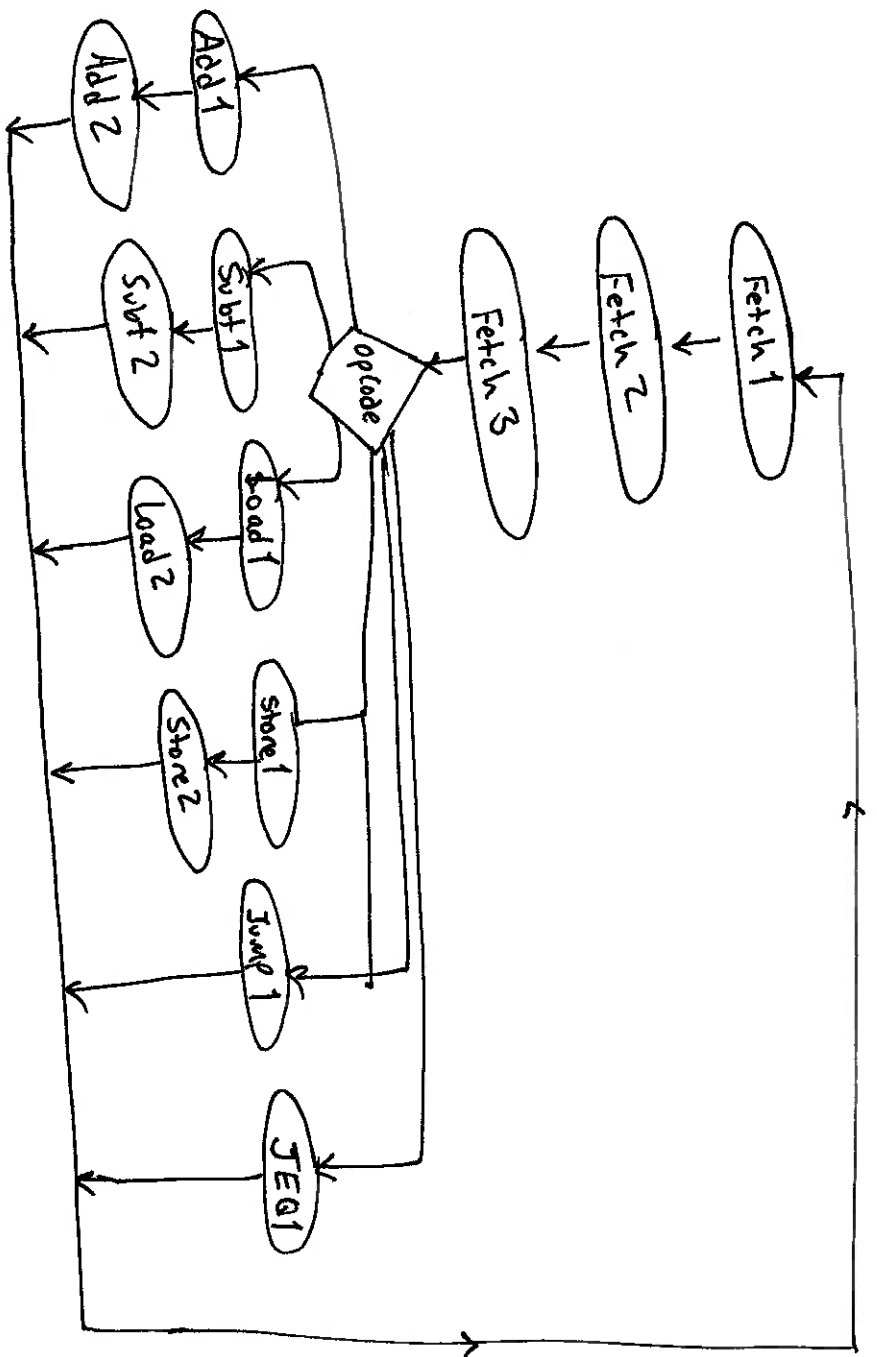
010
Add 1
 • Mem bus
 • Mem read
 • DR load
 $DR \leftarrow m[AR]$
Add 2
 • ALU Sel (00)
 • ALU Bus
 • AC load
 $AC \leftarrow AC + DR$

JEQ 1
 • PC load Z
 • PC INC
 If $(Z=1)$
 $PC \leftarrow DR[4:0]$
 else
 $PC \leftarrow PC + 1$

Store 1
 • AC Bus
 • DR load
 $DR \leftarrow AC$
Store 2
 • DR bus
 • Mem write
 $M[AR] \leftarrow DR$

Sub 1
 • Mem bus
 • Mem read
 • DR load
 $DR \leftarrow m[AR]$
Sub 2
 • ALU Sel (01)
 • ALU Bus
 • AC load
 $AC \leftarrow AC - DR$





MUX

IR/opcode	Function	Address
0000	Load	00100
0001	Store	00116
0100	Add	01000
0111	Subtract	01010
1006	Jump	01100
1001	Jump/2a	01110

MicroCode Memory

operation	Address	Control sig.	Sel	Next Addr
Fetch1	00000	0000	0	00001
Fetch2	00001	0001	0	00010
Fetch3	00010	0010	1	00000
Load1	00100	0011	0	00101
Load2	00101	0100	0	00000
Store1	00110	0101	0	00111
Store2	00111	0110	0	00000
Add1	01000	0111	0	01001
Add2	01001	1000	0	00000
Subt1	01010	1001	0	01011
Subt2	01011	1010	0	00000
Jump1	01100	1011	0	00000
JEQ1	01101	1100	0	00000

Main Memory

Address	Value
00000	IR Add1
00001	IR Add
00010	IR Add
00011	IR Add
00100	IR Add
00101	IR Add
00110	IR Add
00111	IR Add
01000	IR Add
01001	IR Add
01010	IR Add
01011	IR Add
01100	IR Add
01101	IR Add
01110	IR Add
01111	IR Add

Program Entries

IR IR IR A A A A A A
opcode data address

Data

10000	
10001	
10000	
10011	
10100	
10101	
10110	
10111	
11000	
11001	
11010	
11011	
11100	
11101	
11110	
11111	